

Development and validation of icons varying in their abstractness

Mariano García, Albert N. Badre* and John T. Stasko*

Icons are used widely in human–computer interfaces. The level of abstractness–concreteness of an icon and its effect upon performance is of widespread interest. The authors have devised a quantitative measure for abstractness based on the complexity of the icon. They test their metric against subjective judgments of abstractness as identified by two different groups of subjects. After ranking two sets of ‘abstract’ and ‘concrete’ icons, the authors examined how well the icons were matched to the Pascal constructs that they represented. Further experiments were conducted using different groups of subjects to check whether correct matching of the icons with constructs was influenced by context. In summary the authors found that their metric was a good match for subjective measures of abstractness–concreteness. They also found that there is a better identification of concrete icons than abstract icons. Finally, it was shown that context does affect the correct identification of icons.

Keywords: human–computer interaction, interfaces, icons, abstractness–concreteness

Icons are graphic symbols that can be used to represent actions or operations and that are designed to convey ideas or information in a non-verbal manner. The use of icons continues to gain popularity in modern human–computer interfaces. Developing a taxonomy of classification of icons is important because it helps determine which dimensions are relevant in the interpretation of those icons. This will lead to strategies for choosing icons for a particular task. One methodology suggested by Lodding (1983) is to develop a taxonomy where icons are classified by their design and the image functions. We propose another dimension that varies on the abstractness of the representation, varying from a concrete image to an abstract image.

Our domain of interest is the use of icons to represent procedural program-

Mathematics Department, Cayey University College, Cayey, Puerto Rico 00737, USA. Tel: (809) 738-1261. E-mail: m_garcia@cucac@upr1.upr.clu.edu

*College of Computing, Georgia Institute of Technology, Atlanta, GA 30332, USA. Tel: (404) 853-9386. Fax: (404) 853-0673. E-mail:stasko@cc.gatech.edu

Paper submitted: October 1992; revised July 1993

ming language constructs for use in visual programming systems. Concrete icons are ones that represent the constructs at an operational level. Abstract icons are ones which represent the constructs at a higher conceptual level. The metric we present is based on the premise that a higher level of detail in an icon will evoke a more concrete relationship between the icon and the construct from the users' point of view.

Interfaces using icons to represent programming language constructs have been developed which allow users to build programs (Edel, 1988; Glinert and Tanimoto, 1984). Standardizing icons is beneficial because it may lead to the development of icons which may be universally understood. This leads to increased portability of icon-based systems and provides a common frame of reference for the evaluation of these systems.

Rogers (1989) concludes that the use of icons in interfaces will not end soon. She suggests that a taxonomy be developed initially to categorize the different functions that need to be represented at the interface. Research on iconic interfaces has shown that the care with which an interface is developed is more important than the interface style (Potosnak, 1987; Selker *et al.*, 1987; Whiteside *et al.*, 1985). We strove here to develop icon sets that were based on the responses of typical users, which should help ensure future success of any icon-based system to be used by users with similar characteristics. The effect of icons on accuracy and speed in user interfaces has also been the object of study (Kacmar and Olsen, 1991; Shulman *et al.*, 1985). Ease of learning is also a variable that should be taken into account when assessing the effectiveness of an iconic interface.

The level of abstractness of an icon and its effect on performance has been studied in previous experiments. Rogers (1986) had subjects match icons with word-processing commands in order to show that a large variety of command operations could be comprehended successfully in an iconic form. She also developed six categories of concreteness-abstractness. She found that the most effective type of mapping was that of pictorial representations depicting concrete objects directly operated on. She also found that the least effective icons were those which depicted the command operation by analogy. In a study of computer-aided design (CAD) icons, Stammers *et al.* (1989) found superiority for concrete icons in relation to error rates. Rohr and Keppel (1985) had subjects perform text-processing tasks to determine how people handled a set of pictorial symbols in comparison to a verbal command set. They found that the group viewing concrete (functional) icons had fewer help requests than the group viewing abstract (non-functional) icons. Arend *et al.* (1987), however, found that abstract icons were searched and selected much faster than both word commands and representational icons in menu selection for a word-processing task.

Stammers and Hoffman (1991) studied transfer of performance between concrete and abstract sets, finding superiority for concrete icon sets in the error data, but no advantage from previous experience with a concrete set. They also had subjects rank icons using Rogers' (1986) categories of concreteness-abstractness. In an icon appropriateness ranking study, Nolan (1989) found that in general, respondents preferred more concrete icons to more abstract icons.

He also noted that European respondents preferred the more abstract icons. He presumed this was due to their familiarity with pictographic representations as part of everyday life. However, no quantitative data was provided on the comparison of the European and US respondents.

Problem statement

Our research has two goals:

- to develop and apply a metric for icons that allows us to predict the interpretation of icons by subjects;
- to determine if the level of perceived abstractness of an icon has an effect on the ability to identify the icon.

Icons are essential to a number of application domains. We examine icons that can be used in visual programming, the use of graphical symbols rather than the textual commands for representing the constructs of programming languages (Chang, 1987; Shu, 1988). In such a visual environment, programs are constructed by manipulating icons. For these experiments, we have developed icons to represent the constructs of a typical programming language. We seek to compare the interpretation of these icons by two groups of subjects with differing cultural backgrounds, to determine if there is a significant difference between the two groups in interpretations of the icons. We also seek to verify the suitability of the icons selected as the most concrete and abstract to represent the intended programming language constructs. Our interest here is to determine if there is a difference in performance depending on the icon set which is used. Determining the effects of the level of abstractness of the icons used may contribute to the development of more effective visual environments. In order to achieve the two stated goals, we conducted two experiments: **ranking** and **matching**.

Experiment 1: ranking

Definition of metric

We have developed a metric that can be applied to icons in order to characterize their abstractness. The basis of the metric is that symbols which are more complex are perceived as being more concrete. The measure of a particular icon is determined by adding up the number of the following components in each icon:

- closed figures,
- letters,
- open figure (where the figure's outline is not continuous),
- special characters (for example: ?,=),
- horizontal lines,
- vertical lines,
- diagonal lines,

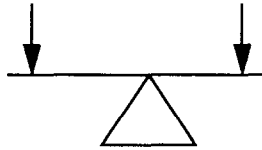


Figure 1. Sample icon that exhibits metric value 6

- arrowheads,
- arcs.

As an example of the metric, the drawing in Figure 1 has a value of 6. This value is obtained by adding up 2 vertical lines, two arrowheads, one horizontal line, and one closed figure.

This metric was developed due to the necessity of classifying icons as being abstract or concrete. Previous experiments had classified icons based on the experimenter's judgements. The advantage of this metric is that it provides a quantification to classify icons, instead of having to rely on subjective interpretation. Various possibilities for the basis of the metric were considered. These included the size of the elements used, the presence or absence of particular elements (circles, rectangles, etc.) and the likeness of arrangements of elements to common figures. Through discussion and refinements we arrived at the metric definition presented at the start of this section.

The metric is not perfect, however. Possible problems with it are the different groupings of components by different subjects and the possibility of creating icons which have similar values of the metric. It is also possible artificially to construct an icon with a great number of components that actually represents an abstract concept. This type of construction should rarely occur in practice, however. All in all, the metric does seem to be a good characterization of abstractness as the following sections will show.

Metric applied to previous experiments

We applied the proposed metric to icons used in four previous experiments that involved icons varying in their level of abstractness. These four experiments were:

- Arend *et al.*, (1987) — word processing;
- Rogers (1986) — word processing;
- Rohr and Keppel (1985) — word processing;
- Stammers *et al.* (1989) — CAD.

The icons used in these experiments were developed by the experimenters and then included in their interfaces. In all these experiments, our metric had a higher value for the icon classified as concrete by the experimenters than for the icon classified as abstract by the experimenters. The tables that follow show the value of the metric for the icons classified as abstract and concrete by the experimenters. In the Rohr and Keppel article, the sets were classified as functional and non-functional.

Table 1. Arend *et al.*

Command	Metric value of abstract icon	Metric value of concrete icon
Scroll down	4	11
Scroll back	4	11
Store text	2	11
Delete word	3	12
Replace word	5	14
Search for word	2	14
Skip to next word	2	12
Skip to previous word	2	12
Move a sentence	2	9
Delete line	4	14
Insert line	3	13
Justify text	3	14

Table 2. Rohr and Keppel

Command	Metric value of abstract icon	Metric value of concrete icon
Erase	1	2
Print	3	4
Move	1	7
Replace	4	9

Table 3. Rogers

Command	Metric value of abstract icon	Metric value of concrete icon
Go to bottom of text	3	8
Insert a line	2	6
Delete a block of text	2	4
Save a file	1	4
Quit	2	5

Metric applied to icons used for Pascal constructs

We developed icons that varied in their level of abstractness and that represented the constructs of a procedural programming language. The nine constructs represented were assignment, begin-end, case, for, if-then-else, input (readln), output (writeln), repeat, and while. These constructs are sufficient to solve the first few sets of problems encountered in an introductory programming course in Pascal. Reilly (1989) used a similar set of constructs in a text designed to teach programming in Pascal.

We considered various symbols for each construct. These symbols varied in

Table 4. Stammers *et al.*

Command	Metric value of abstract icon	Metric value of concrete icon
Create drawing	1	9
Retrieve drawing	5	7
Plot options	5	7
Create private file	5	12
Journal recovery	2	7
Compress drawing file	6	8
Compact drawing file	1	16
Compact private file	6	17
Execute ext prog	11	23
Examine archive	1	11
Add archive entry	1	12
Modify archive	6	14
Retrieve parametric symbol	4	7
Quit	2	7

their level of abstractness. Four candidate icons were selected for each construct. These four icons all had different values of the metric. Figures 2, 3, and 4 illustrate the icons we selected for each construct and the values of the metric for each icon.

Methodology

Method

After developing these icons, we were prepared to present the icons with differing metric values to subjects in order to determine how they would be perceived. Two different groups of subjects were used. The first group of subjects were undergraduates taking a psychology course at an engineering school. They were volunteers and received course credits for participating in the experiment. This group comprised 23 subjects. The second group of subjects were undergraduates who had taken or were taking an introductory programming course in Pascal at a liberal arts school in Puerto Rico. The subjects were volunteers and received course credits for participating. This second group comprised 18 subjects. A questionnaire was administered to the subjects who participated in the experiment to collect information on age, sex, year in school, major field of study, exposure to computers and/or programming courses, and math courses taken in high school.

Materials

The materials used for this experiment were nine sets of four 4×6 index cards, each showing a different icon that represents one of the nine programming language constructs. The four icons in each set had a different value of the proposed metric.

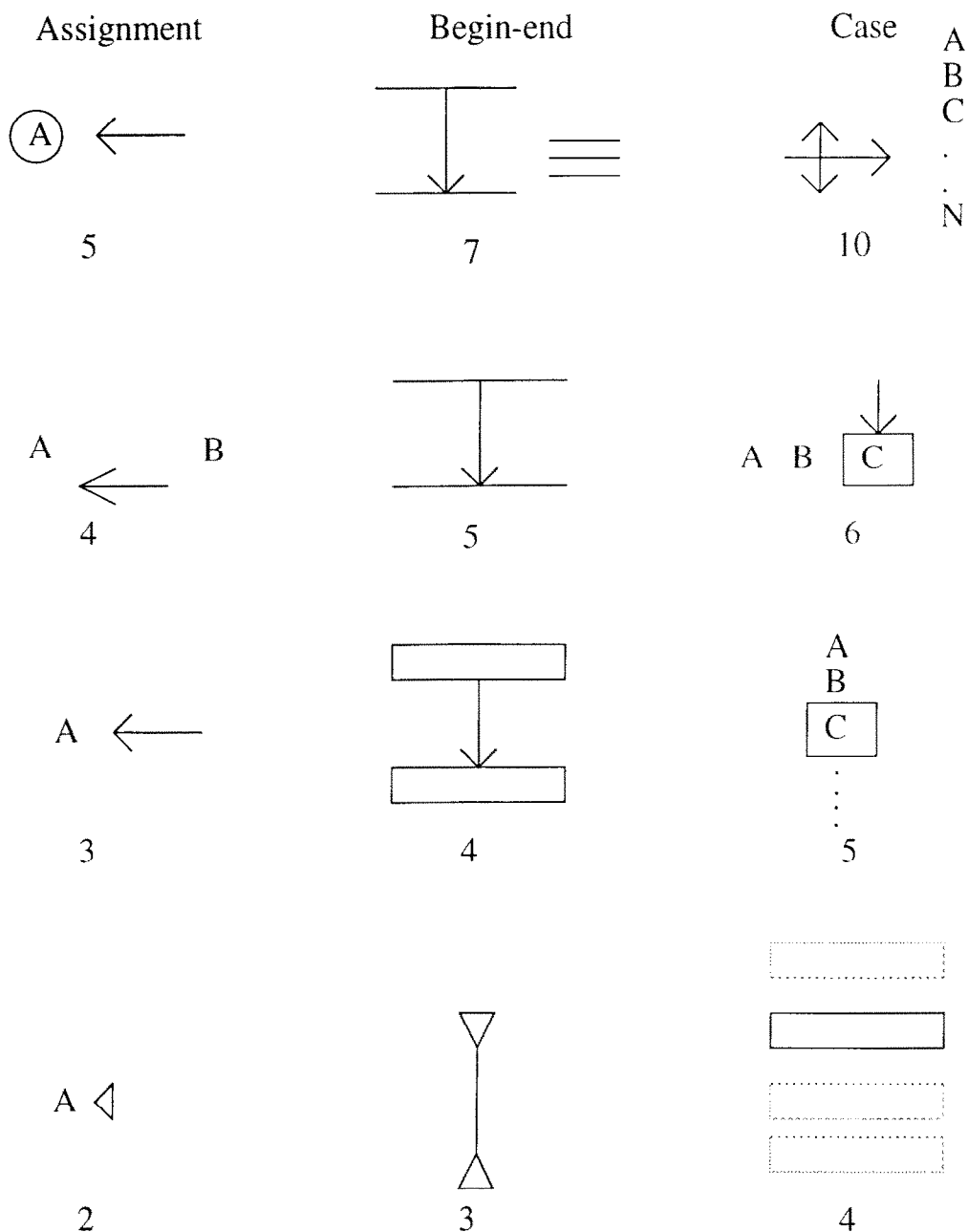


Figure 2. Values of metric for 'Assignment', 'Begin-end' and 'Case' icons

Procedure

Subjects ranked the icons individually, with the experimenter present at all times during the experiment. After the subjects filled out the questionnaire, the experimenter read the instructions to the subjects. The instructions presented

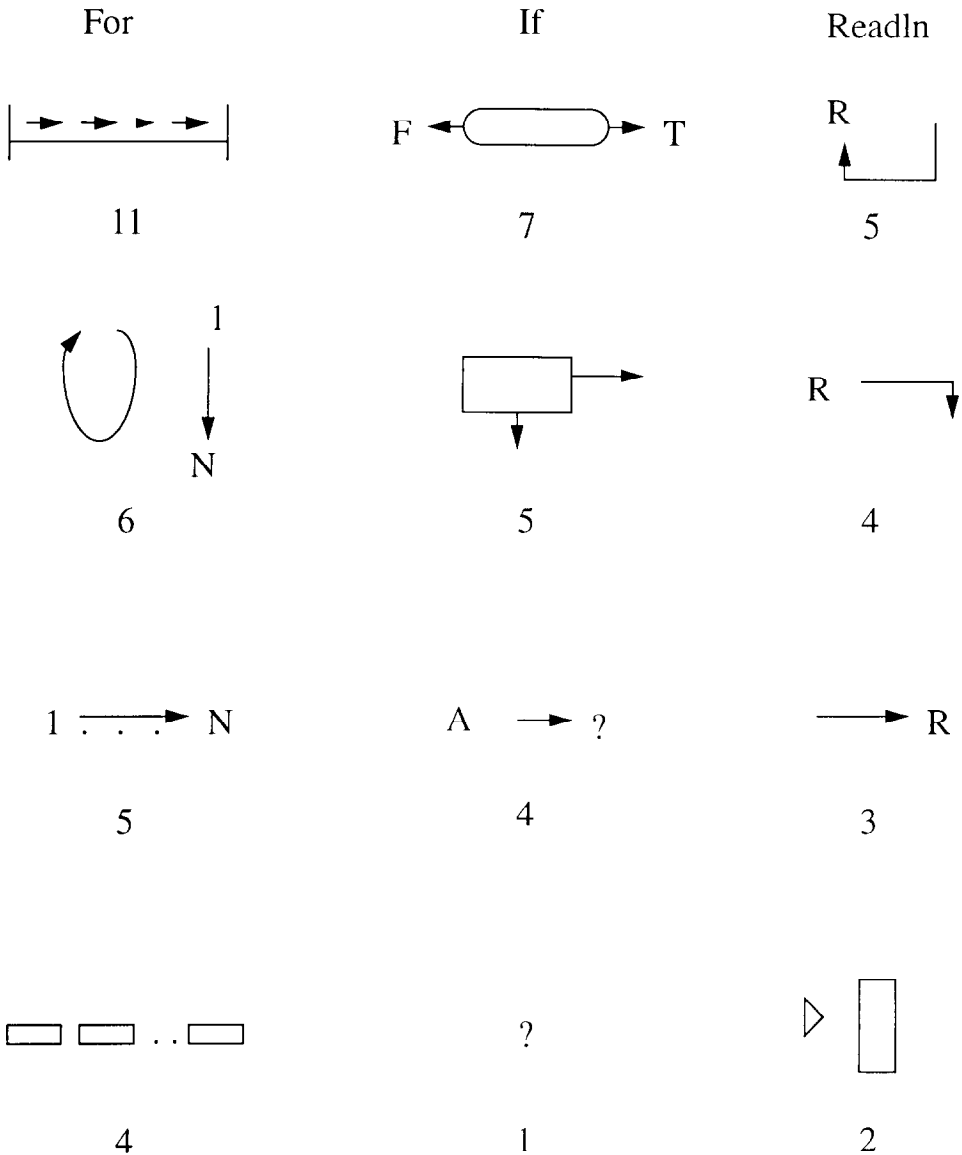


Figure 3. Values of metric for 'For', 'If' and 'Readln' icons

examples of a concrete icon and an abstract icon. These sample figures shown to the subjects were taken from Lodding's (1983) article. The icons should not have biased the subjects' interpretations of the icons in our experiments, because Lodding's icons are not related to programming. After answering any questions from the subjects, the experimenter presented the set of four index cards for each programming language construct and asked the subject to rank the icons from the most concrete icon to the most abstract icon. The subjects also

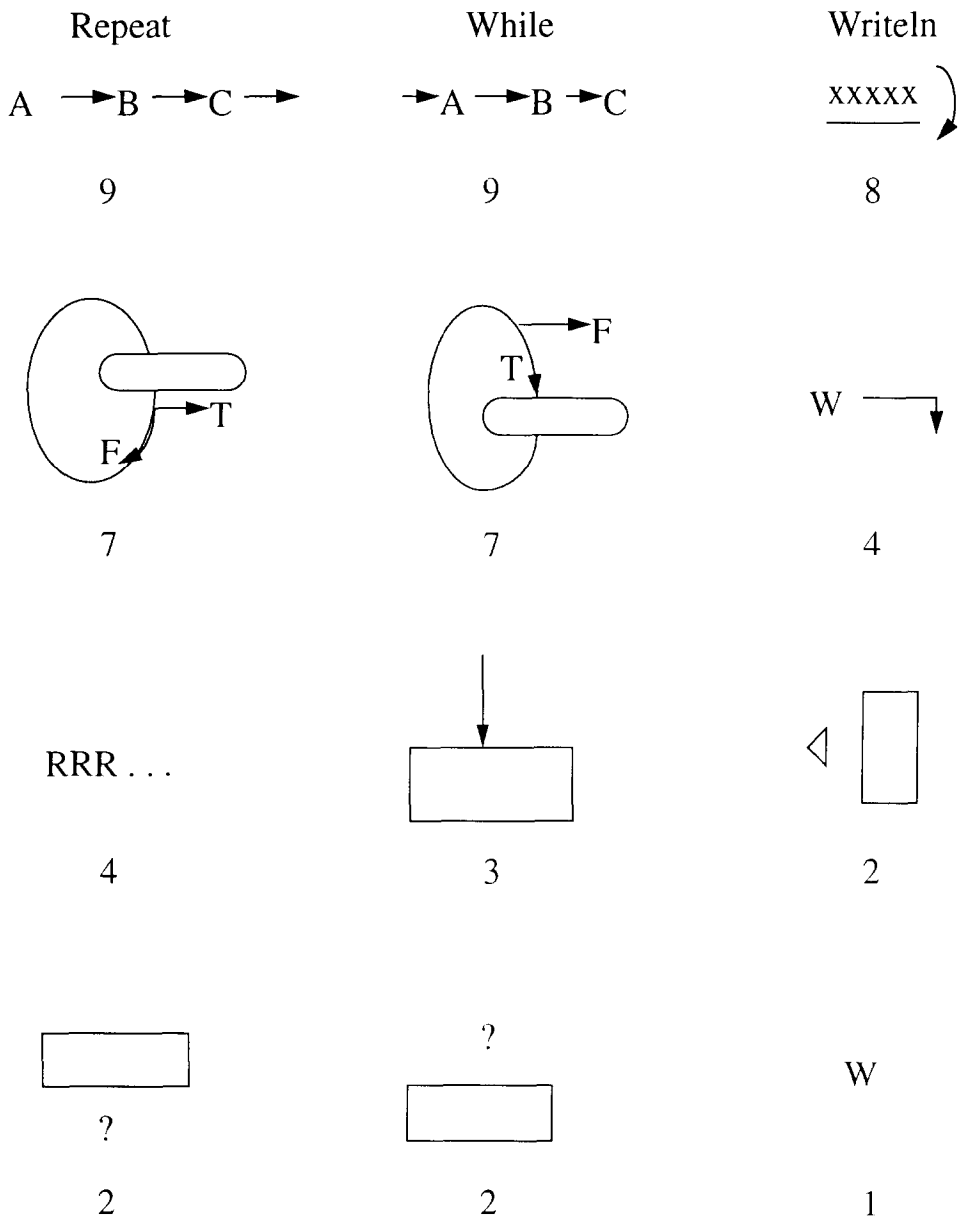


Figure 4. Values of metric for 'Repeat', 'While' and 'Writeln' icons

classified each icon as being concrete or abstract. The order of the four icons was randomized by shuffling each group of index cards before showing them to the subjects. The order of the constructs themselves was randomized by shuffling nine index cards (one from each of the construct sets) before the experiment began.

Table 5. Metric values and rankings for each construct — first group

Construct		Characterization			
Assignment	Value of metric	2	3	4	5
	Ave rank	3.65	2.91	1.26	2.17
	Median	4	3	1	2
Begin-end	Value of metric	3	4	5	7
	Ave rank	3.39	1.83	2.74	2.04
	Median	4	2	3	2
Case	Value of metric	4	5	6	10
	Ave rank	3.22	2.13	1.30	3.35
	Median	4	2	2	3
For	Value of metric	4	5	6	11
	Ave rank	2.87	1.91	2.17	3.04
	Median	3	2	2	3
If	Value of metric	1	4	5	7
	Ave rank	3.39	2.56	1.78	2.26
	Median	4	3	1	2
Readln	Value of metric	2	3	4	5
	Ave rank	3.43	2.00	2.09	2.48
	Median	4	2	2	2
Repeat	Value of metric	2	4	7	9
	Ave rank	3.52	1.78	2.17	2.52
	Median	3	2	2	3
While	Value of metric	2	3	7	9
	Ave rank	3.30	2.69	2.22	1.78
	Median	4	3	1	2
Writeln	Value of metric	1	2	4	8
	Ave rank	2.04	3.56	1.87	2.52
	Median	2	4	2	3

Results and discussion

Ranking of icons by first group

We ranked the icons for each construct from the most concrete to the most abstract based on the subjects' responses. These rankings were analysed using non-parametric statistical techniques. A first analysis determined if the icons were perceived as being different from one another. A Friedman test for ranked data was used, with a null hypothesis that all the icons were perceived as being the same. This null hypothesis was rejected for all constructs in the engineering school group with all α^* below 0.02.

In the experiment, the subjects were instructed to rank the icons from the most concrete (1) to the most abstract (4). This meant that the icons with lower average rank values were interpreted as being more concrete. This may be a bit confusing because it is reversed from our metric scheme where the icons with higher metric values were projected to be interpreted as being more concrete. The rankings for each construct are shown in Table 5.

Of interest here is the performance of the proposed metric. The metric did not

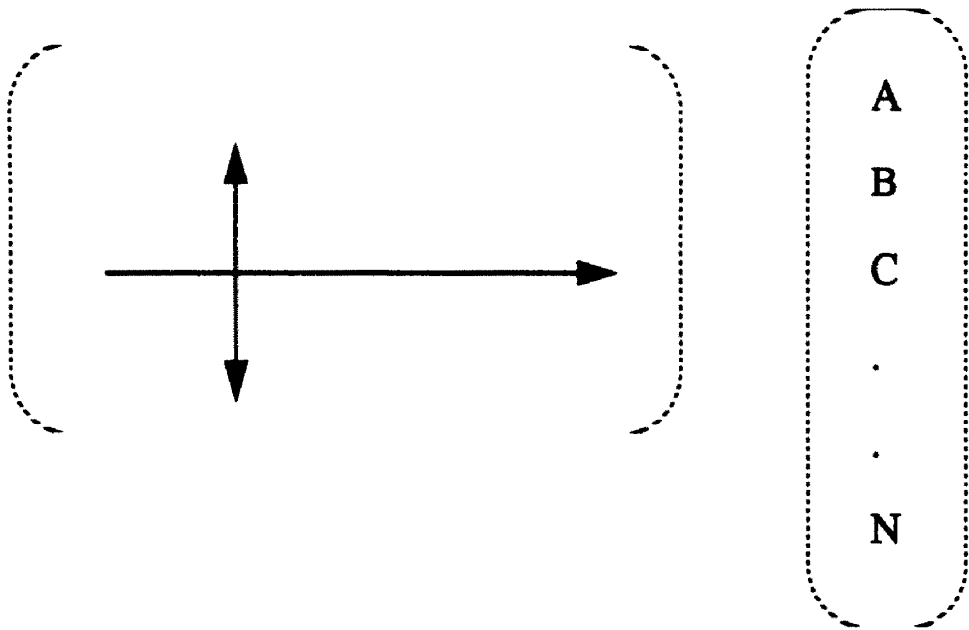


Figure 5. 'Case' icon, metric value 10

provide a perfect prediction of the rankings obtained. In the group of subjects from the engineering school, the value of the metric was higher for the icon perceived as the most concrete icon than the icon perceived as the most abstract icon for all except two (Case, For) of the nine constructs.

The icon perceived as the most abstract for the 'Case' statement has a value of 10 for the metric (see Figure 5). The value was based on its components: 4 letters, 1 enclosed ellipsis, 1 vertical line, 1 horizontal line, and 3 arrowheads. What seems to have happened here is that the icon was perceived as consisting of two simple components: the letters with the ellipsis and the set of arrowheads. If each entity is perceived as one unit, this icon would have a metric value of 2.

In the 'For' icon, the metric had a proposed value of 11 for the icon perceived as most abstract (see Figure 6). This was based on 5 horizontal lines, 2 vertical lines, and 4 arrowheads. It seems that the set of four arrowheads was perceived as an entity, perhaps because it was enclosed within a figure. If that entity was regarded as one unit, this figure would have a metric value of 4. Examples such

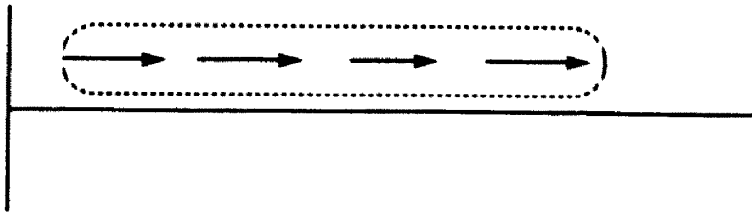


Figure 6. 'For' icon, metric value 11

as these illustrate the shortcomings of the metric like those we mentioned earlier. Overall, however, the metric does function as a good quantitative measure instead of having to rely on subjective evaluations of icons.

In the 'Writeln' construct, the symbol with a metric value of 1 (the letter "W") was perceived as being fairly concrete. What seems to have happened is that the letter used evoked a concrete relationship with the word 'Writeln'. If a different letter had been used, it probably would not have been perceived as being so concrete.

The metric predicted the most concrete interpretation in only one construct (While). In the 'Assignment' icon, the 'Case' icon and the 'For' icon, the presence of separated letters seems to have caused the icons to be perceived in a more concrete manner. In the 'Readln' and 'Writeln' icons, the presence of an arrow going from left to right seems to have caused the icons to be perceived in a more concrete manner. In the 'Begin-end' and 'If' icons, the presence of rectangles appears to have caused a more concrete perception.

Ranking of icons by second group

This experiment was repeated with the subjects from the liberal arts school in Puerto Rico. Again, the initial analysis was to determine if the icons had been perceived as being different from each other using a Friedman test. With this group, we were not able to reject the null hypothesis at the 0.05 level for the icons used for 'If' and 'Begin-end'. It would have been necessary to use an α level of 0.06 to be able to say that the icons were perceived as being different from each other by this group. The results are summarized in Table 6.

In this group of subjects, the icon that was perceived as the most concrete (with the lowest average rank) had a higher metric value than the icon perceived as the most abstract for all nine constructs. The metric predicted the icon perceived as the most abstract in eight of the nine constructs. Again the metric predicted the icon perceived as the most concrete in only one construct (While). The metric did a slightly better job of characterizing the perceived abstractness of the icons with this group than it did with the previous group.

Comparison of groups

We evaluated the results from both groups using a two-sample *t*-test to compare the means of the rankings of each icon by the two groups. We also used the Mann-Whitney U-test to compare the relative frequency distributions of the rankings of the icons. The null hypothesis used is that the two samples come from populations that are similar.

The similarities between the two groups in their responses to the icons is evidenced by the high *p* values that the *t*-test and the Mann-Whitney test calculate in most of the cases. In only 4 of the 36 (4 icons each for 9 constructs) cases were we able to reject the null hypothesis at the 0.05 α level. These four cases were the icon with a metric value of 2 for 'Assignment', the icons with metric values of 6 and 10 for 'Case', and the icon with metric value of 11 for 'For'.

Also of interest is the fact that the icon that was perceived as the most abstract for a construct was the same for both groups in seven of the nine constructs. The constructs that did not elicit the same icon for the most abstract interpretation

Table 6. Metric values and rankings of each construct — second group

Construct		Characterization			
Assignment	Value of metric	2	3	4	5
	Ave rank	3.11	2.94	1.61	2.33
	Median	3	3	1	2
Begin-end	Value of metric	3	4	5	7
	Ave rank	3.00	1.94	2.56	2.50
	Median	3.5	1.5	3	2.5
Case	Value of metric	4	5	6	10
	Ave rank	3.50	2.08	1.89	2.56
	Median	3	2	1	4
For	Value of metric	4	5	6	11
	Ave rank	3.39	1.94	2.22	2.44
	Median	4	2	2	3
If	Value of metric	1	4	5	7
	Ave rank	3.06	2.50	1.94	2.50
	Median	3.5	2.5	2	2
Readln	Value of metric	2	3	4	5
	Ave rank	3.61	1.83	2.22	2.33
	Median	4	1.5	2	2
Repeat	Value of metric	2	4	7	9
	Ave rank	3.39	1.94	2.22	2.44
	Median	4	1.5	2.5	2
While	Value of metric	2	3	7	9
	Ave rank	3.44	2.22	2.17	2.17
	Median	4	2	1.5	2
Writeln	Value of metric	1	2	4	8
	Ave rank	1.94	3.67	1.72	2.67
	Median	2	4	2	3

were 'Case' and 'For'. The icon that was perceived as the most concrete was the same for both groups in all nine constructs (in one of the groups there was a tie in the 'While' construct).

Experiment 2: matching with perceived icons

In an effort to validate the icons used to represent the programming language constructs, a second experiment was carried out with a different group of subjects from the engineering school. We had subjects match the icons perceived as most concrete and most abstract in the previous experiment with the original programming language constructs. The icons that obtained satisfactory identification percentages were to be included in icon sets for future experiments.

Method

Twenty-four subjects were presented with a display on a computer terminal that allowed them to match icons with one of ten alternatives (the nine

programming constructs plus 'none'). None of the subjects in this experiment had participated in the first experiment. As before, the subjects were volunteers taking a psychology course and received course credit for their participation. To qualify for this experiment, a student must have taken a Pascal course previously. All subjects matched the set of nine abstract icons and the set of nine concrete icons. The order in which the subjects saw the sets was randomized. The order of the icons and the words also was randomized as was the location (top-half or bottom-half) of the icons and the words.

Materials

This experiment was carried out on a Macintosh II computer using the SRGP graphics package (Foley, 1990).

Procedure

Subjects matched icons with constructs individually, with the experimenter present at all times. Subjects were read instructions and asked if they had any questions. If there were no questions, the subjects were presented with a display in which they used a mouse to select an icon and then select the construct they thought corresponded to the icon. Both the icon selected and the construct selected were erased from the screen after the subject selected the construct. If the subject selected 'none' instead of a construct, the icon would be erased from the screen, but the word none would remain on the screen. This process continued until the subject had seen all the icons in one of the sets. The screen would then be filled again with the second set of icons and the entire process repeated a second time.

Results

The percentages of correct identifications for the icons in the concrete set and the abstract set are shown in Tables 7 and 8. When analysing the results for the concrete icon set, we note satisfactory identification percentages for all icons except the icon used for the 'While' construct. The 'While' icon was selected as 'none' as the most frequent response and was identified as 'For' as often as 'While'. This caused us to exclude this particular 'While' icon (whose metric value was 9) from future sets. We substituted for it with the 'While' icon which had been perceived as second most concrete (with a metric value of 7).

In the results for the abstract icon set, we notice that the percentages of correct identification are lower for almost all icons when compared to the percentages for the concrete icon set. The 'For' icon was identified as 'Begin-end' almost as frequently as 'For'. The 'Readln' icon was confused with the 'Writeln' icon.

Discussion

One of the objectives of these experiments was to build a concrete icon set and an abstract icon set to represent programming language constructs. We needed to select one icon for each construct for inclusion in the sets. We had expected to use the icons with the lowest value of the metric for the abstract set and the icons with the highest value of the metric for the concrete set. These results caused us to modify the icon that is to be used for 'Writeln' in the abstract icon

Table 7. Matches between concrete icons and constructs

Construct	Assign	Begin	Case	Subject selection			Repeat	While	Write	None
				For	If	Read				
Assign	16	0	0	0	0	1	4	0	0	3
%	67	0	0	0	0	4	17	0	0	13
Begin	0	15	1	0	2	0	0	1	0	5
%	0	63	4	0	8	0	0	4	0	21
Case	3	0	14	0	1	3	0	0	0	3
%	13	0	58	0	4	13	0	0	0	13
For	1	1	0	19	0	0	1	2	0	0
%	4	4	0	79	0	0	4	8	0	0
If	1	0	1	0	20	0	0	0	0	2
%	4	0	4	0	83	0	0	0	0	8
Readln	0	0	0	0	0	17	1	0	1	5
%	0	0	0	0	0	71	4	0	4	21
Repeat	0	0	0	0	0	0	16	2	2	4
%	0	0	0	0	0	0	67	8	8	17
While	1	1	2	3	1	2	0	3	1	10
%	4	4	8	13	4	8	0	13	4	42
Writeln	0	0	0	1	0	0	0	4	18	1
%	0	0	0	4	0	0	0	17	75	4

Table 8. Matches between abstract icons and constructs

Construct	Assign	Begin	Case	Subject selection			Repeat	While	Write	None
				For	If	Read				
Assign	15	0	3	0	0	2	1	1	2	0
%	64	0	13	0	0	8	4	4	8	0
Begin	0	14	0	1	0	1	3	1	0	4
%	0	58	0	4	0	4	13	4	0	17
Case	2	1	15	2	0	1	2	1	0	0
%	8	4	63	8	0	4	8	4	0	0
For	1	8	0	10	0	0	1	1	3	0
%	4	33	0	42	0	0	4	4	13	0
If	0	0	1	0	16	2	0	0	1	4
%	0	0	4	0	67	8	0	0	4	17
Readln	2	0	0	1	1	8	0	2	8	2
%	8	0	0	4	4	33	0	8	33	8
Repeat	0	0	1	1	1	1	10	3	1	6
%	0	0	4	4	4	4	42	13	4	25
While	1	0	1	1	3	1	0	11	1	5
%	4	0	4	4	13	4	0	46	4	21
Writeln	1	0	0	1	0	7	4	1	6	4
%	4	0	0	4	0	29	17	4	25	17

set. The initial abstract icon had an arrowhead leaving a rectangle from the left side. This arrowhead was changed to the right side to try to differentiate this icon a little more from the 'Readln' icon.

We also wanted to compare the results obtained in a matching task using concrete icons and abstract icons. The identification of the icons was much better with the concrete icon set than with the abstract icon set.

Experiment 2(a): matching in isolation

We were interested in determining if the icons would be perceived in a

different manner if they were presented in isolation. The same icon sets were used, but only one icon was viewed at a time.

Method

Twenty subjects were presented with a display on a computer terminal that allowed them to match an icon with one of ten alternatives (the nine constructs plus 'none'). None of these subjects had participated in the previous matching experiment. They were volunteers taking an introductory programming course at the engineering school and received course credits for their participation.

Materials

This experiment was carried out on a Sun Sparc station with a Tektronix terminal using the SRGP graphics package.

Procedure

Again, subjects matched icons with constructs individually, with the experimenter present at all times. Subjects were read instructions and asked if they had any questions. Subjects were presented with a display that included one icon and the ten alternatives. Subjects used a mouse to select the construct they thought corresponded to the icon. After selecting the construct, the icon would be erased, but all constructs (and the word 'none') remained on the screen for the next icon. This process continued until the subjects had seen all the icons in the abstract icon set and the concrete icon set.

Results

The percentages of correct identification were fairly consistently lower when the icons were viewed in isolation as compared to having a full set of icons available. In the concrete icon set, there were three cases ('Assignment', 'Repeat', and 'While') where the percentages were slightly higher for the icon in isolation. In the abstract icon set, there were two cases ('Assignment' and 'Begin-end') which had slightly higher percentages for the icon in isolation.

The concrete icons still had a better percentage of correct identification than the abstract icons. When comparing the percentage of correct identification for the icons in isolation, we notice greater percentages for the concrete icon as compared to the abstract icon except in the 'Begin-end' construct. Tables 9 and 10 show the percentages of correct identification for the icons in the concrete icon set and the abstract icon set.

Discussion

Our conjecture here is that icons should be used contextually. People seem to recognize and discriminate between icons better when in a meaningful situation. In a visual programming environment, users will likely have access to a whole set of icons rather than individual icons in isolation. The contextual use of icons may be an important consideration in the design of icons to represent programming language constructs.

Table 9. Correct matches between constructs and concrete icons in isolation

Construct	Assign	Begin	Case	Subject selection			Repeat	While	Write	None
				For	If	Read				
Assign	14	0	0	1	3	0	2	0	0	0
%	70	0	0	5	15	0	10	0	0	0
Begin	7	6	0	1	1	0	2	2	0	1
%	35	30	0	5	5	0	10	10	0	5
Case	3	0	9	1	0	3	1	2	0	1
%	15	0	45	5	0	15	5	10	0	5
For	1	0	2	11	0	0	3	2	0	1
%	5	0	10	55	0	0	15	10	0	5
If	1	2	2	0	9	1	0	0	0	5
%	5	10	10	0	45	5	0	0	0	25
ReadIn	1	0	0	1	0	3	8	1	2	4
%	5	0	0	5	0	15	40	5	10	20
Repeat	0	0	0	0	0	0	15	0	0	5
%	0	0	0	0	0	0	75	0	0	25
While	0	0	1	1	10	0	3	4	0	1
%	0	0	5	5	50	0	15	20	0	5
Writeln	0	1	0	0	0	0	0	9	8	2
%	0	5	0	0	0	0	0	45	40	10

Table 10. Correct matches between constructs and abstract icons in isolation

Construct	Assign	Begin	Case	Subject selection			Repeat	While	Write	None
				For	If	Read				
Assign	13	0	1	0	1	2	1	0	0	2
%	65	0	5	0	5	10	5	0	0	10
Begin	1	13	0	0	1	0	0	1	0	4
%	5	65	0	0	5	0	0	5	0	20
Case	0	0	8	6	0	2	1	0	0	3
%	0	0	40	30	0	10	5	0	0	15
For	1	1	4	5	0	1	3	1	0	4
%	5	5	20	25	0	5	15	5	0	20
If	3	0	1	0	8	3	0	0	0	5
%	15	0	5	0	40	15	0	0	0	25
ReadIn	9	1	1	1	0	0	0	1	1	6
%	45	5	5	5	0	0	0	5	5	30
Repeat	2	0	2	2	3	4	0	0	0	7
%	10	0	10	10	15	20	0	0	0	35
While	2	0	2	0	5	4	0	0	0	7
%	10	0	10	0	25	20	0	0	0	35
Writeln	5	1	0	1	1	1	0	0	2	9
%	25	5	0	5	5	5	0	0	10	45

Experiment 2(b): matching with measured icons

In the next experiment, we proceeded to have subjects match the icons with the highest metric value and the lowest metric value for each construct. We wanted to compare the identifications of these icons with those of the previous experiment which were subjectively perceived to be the most concrete and most abstract icons.

Method

Seventeen subjects were presented with a display on a computer terminal that

Table 11. Matches between constructs and measured, concrete icons

Construct	Assign	Begin	Case	Subject selection			Repeat	While	Write	None
				For	If	Read				
Assign	10	1	0	0	0	5	0	1	0	0
%	59	6	0	0	0	29	0	6	0	0
Begin	1	9	2	0	0	0	0	2	2	1
%	6	53	12	0	0	0	0	12	12	6
Case	0	1	6	3	0	0	1	0	1	5
%	0	6	35	18	0	0	6	0	6	29
For	0	6	1	4	0	2	2	1	0	1
%	0	35	6	24	0	12	12	6	0	6
If	2	0	1	2	11	0	0	0	0	1
%	12	0	6	12	65	0	0	0	0	6
ReadIn	0	0	1	0	0	4	9	1	0	2
%	0	0	6	0	0	24	53	6	0	12
Repeat	2	0	1	2	1	0	3	2	4	2
%	12	0	6	12	6	0	18	12	24	12
While	1	0	0	1	1	3	1	5	1	4
%	6	0	0	6	6	18	6	29	6	24
Writeln	0	0	1	1	1	2	1	2	8	1
%	0	0	6	6	6	12	6	12	47	6

allowed them to match the icons with one of the programming language constructs or the 'none' choice. None of the subjects had participated in any of the previous experiments. They were volunteers taking an introductory programming course and received course credits for their participation. The location and order of the icons and the words was again randomized.

Materials

This experiment was carried out on a Sun Sparc station with a Tektronix terminal using the SRGP graphics package.

Procedure

Once more, subjects matched icons with constructs individually, with the experimenter present at all times. Subjects were read instructions and asked if they had any questions. They then proceeded to see a display containing the nine icons in an icon set and the ten alternatives. They used a mouse to select an icon and the construct they thought corresponded to that icon.

Results

We first analysed the results obtained with the measured concrete icon set. Our main interest here was the percentages of correct identification of the icons that were different from the icon set used in the original matching experiment. We found that for these eight icons, the percentage of correct identifications was lower for the *measured* most concrete icons when compared to the *perceived* most concrete icons. The results are shown in Table 11.

In the results for the abstract icon set, we were interested in the icons that were different from the icon set used in the original experiment based on the subjects' perception. In two of the three cases (Case and For) we found a lower percentage of correct identification for the measured icons as compared to the

Table 12. Matches between constructs and measured, abstract icons

Construct	Assign	Begin	Case	Subject selection			Repeat	While	Write	None
				For	If	Read				
Assign	13	0	0	1	0	1	0	0	1	1
%	76	0	0	6	0	6	0	0	6	6
Begin	0	11	1	0	0	2	0	0	0	3
%	0	65	6	0	0	12	0	0	0	18
Case	1	1	9	1	1	0	3	0	0	1
%	6	6	53	6	6	0	18	0	0	6
For	0	4	1	5	2	1	1	0	0	3
%	0	24	6	29	12	6	6	0	0	18
If	0	0	0	1	11	3	0	0	0	2
%	0	0	0	6	65	18	0	0	0	12
Readln	2	1	2	3	2	4	0	1	1	1
%	12	6	12	18	12	24	0	6	6	6
Repeat	0	0	0	2	0	0	12	1	1	1
%	0	0	0	12	0	0	71	6	6	6
While	0	0	0	0	1	1	1	12	0	2
%	0	0	0	0	6	6	6	71	0	12
Writeln	0	0	0	0	0	0	0	2	14	1
%	0	0	0	0	0	0	0	12	82	6

perceived icon. In the third construct (Writeln) the percentage was higher for the measured icon as compared to the perceived icon. We believe this was due to the use of the letter "W" which helped identify the icon as being Writeln. The results are given in Table 12.

Discussion

We found that subjects performed better with the icon sets that were based on the subjective abstractness rankings than with the icon sets that were based on the metric value. This suggests that the icon set that will be used in a future visual programming environment should be based on the icon sets which take into account subjects' rankings.

Conclusions

We have presented a metric for characterizing the relative abstractness of icons. The proposed metric agreed very closely with the classification assigned in previous studies carried out by other experimenters. In our experiments, the metric matched the subjective interpretation of abstract icons well, and of concrete icons moderately well. The discrepancies noted in our set of experiments seem to be caused by the perception of groups of components as a single entity. We considered reformulating the metric to take into account the possible perception of groups of elements but decided against this. The reason for this decision was that it would be very difficult to anticipate which groups of elements would be perceived as single entities. There may be considerable variation between subjects in grouping elements, and thus this metric would again be somewhat subjective. We decided to maintain the metric in its original form even though the metric can sometimes generate an assessment which we would not judge as 'correct'. Nevertheless, the metric does function as a good

bias-free quantitative measure of icon abstractness where only subjective assessments existed before.

We also presented the results of two experiments. The first experiment selected icons to represent programming language constructs. The results obtained in this experiment suggest that the two groups studied are identically distributed around their median and mean responses to the icons used. We note that the agreement is stronger with the concrete icons than with the abstract icons. We interpret this to suggest that concrete icons may be preferable to abstract icons in tasks that will be performed by users with different cultural backgrounds.

The second experiment matched the icons with the original constructs. We note there is better identification of the icons with the concrete set than with the abstract set. This may be interpreted to reinforce the choice of concrete icons instead of abstract icons in tasks that will be performed by users with different cultural backgrounds. We also matched the icons in isolation with the programming language constructs. The identification percentages were lower in almost all of the cases. We interpret this to mean that the context in which the icon is viewed is important for identification. We also had subjects match the icons based on the metric with the original construct. In ten of the eleven icons which were different from the subjectively perceived icon sets, the identification percentage was greater with the icon from the perceived sets. This suggests taking into account responses from typical users when selecting icons for a task.

The icons developed in this series of experiments will be used in a visual programming environment designed for use by students taking an introductory Pascal course. Experiments will be conducted with the environment to determine the effect of the icon set used (concrete or abstract) on the ease of learning a procedural programming language (in this case, Pascal). Results from that experiment should provide insight into the effect of differing icon sets used in visual programming environments.

References

- Arend, U., Muthig, K.-P., and Wandmacher, J. (1987) 'Evidence for global superiority in menu selection by icons' *Behav. Inf. Technol.* 6, 411-426
- Chang, S.-K. (1987) 'Visual languages: a tutorial and survey' *IEEE Softw.* 4, 29-39
- Edel, M. (1988) 'The Tinkertoy graphical programming environment' *IEEE Trans. Softw. Eng.* 8, 1110-1115
- Foley, J.D., van Dam, A., Feiner, S.K. and Hughes, J.F. (1990) *Computer Graphics Principles and Practice* Addison-Wesley
- Glinert, E. and Tanimoto, S. (1984) 'Pict: An interactive graphical programming environment' *Computer* 17, 7-25
- Kacmar, H.R. and Olsen, K.A. (1991) 'Icons in man-machine communication' *Behav. Inf. Technol.* 10, 443-457
- Lodding, K.N. (1983) 'Iconic interfacing' *IEEE Comput. Graph. Appl.* 3, 11-20
- Nolan, P. (1989) 'Designing screen icons: ranking and matching studies' in *Proc. Human Factors Soc. 33rd Meeting*
- Potosnak, K. (1987) 'Do icons make user interfaces easier to use' *IEEE Softw.* 1, 97-99

- Reilly, E.D. and Federighi, F.D.** (1989) *Pascalgorithms* Houghton Mifflin
- Rogers, Y.** (1986) 'Evaluating the meaningfulness of icon sets to represent command operations' in **Harrison, M.D. and Monk, A.F. (eds)** *Human Factors in Organizational Design and Management* Cambridge University Press
- Rogers, Y.** (1989) 'Icons at the interface: their usefulness' *Interacting with Computers* **1**, 1, 105–117
- Rohr, G. and Keppel, E.** (1985) 'Iconic interfaces: where to use and how to construct' in **Hendrick, H.W. and Brown, O. (eds)** *Human Factors in Organizational Design and Management* Elsevier Science Publishers
- Selker, T., Wolf, C., and Koved, L.** (1987) 'A framework for comparing systems with visual interfaces' in **Bullinger, H.-J. and Shackel, B. (eds)** *INTERACT '87* Elsevier Science Publishers
- Shu, N.** (1988) *Visual Programming* Van Nostrand
- Shulman, H.G., Shute, S., and Weissman, O.** (1985) 'Icons versus names as command designators in text editing' in *Proc. Int. Conf. Cybernetics and Society* IEEE Press
- Stammers, R.B., George, D.A., and Carey, M.S.** (1989) 'An evaluation of abstract and concrete icons for a CAD package', in **Megaw, E.D. (ed)** *Contemporary Ergonomics* Taylor and Francis
- Stammers, R.B. and Hoffman, J.** (1991) 'Transfer between icon sets and ratings of icon concreteness and appropriateness' in *Proc. Human Factors Society 35th Ann. Meeting*
- Whiteside, J., Jones, S., Levy, P., and Wixon, D.** (1985) 'User performance with command, menu and iconic interfaces' in *Proc. CHI85* ACM Press